SentryScan: Assignment 5.1 — Cryptographic Message Authentication & Security Gate

Course: 21CSE525T — Agent Technology
Assignment: 5.1 — Cryptographic Message Authentication & Security Gate
Student: R Ganesh [RA2212704010031]
Repository: https://github.com/drgeekg/SentryScan
Target Branch: `assignment/5.1-authentication`
Git Tag: `assignment-5.1`
Status: Completed & Verified

---

1. Executive Summary & Theoretical Context

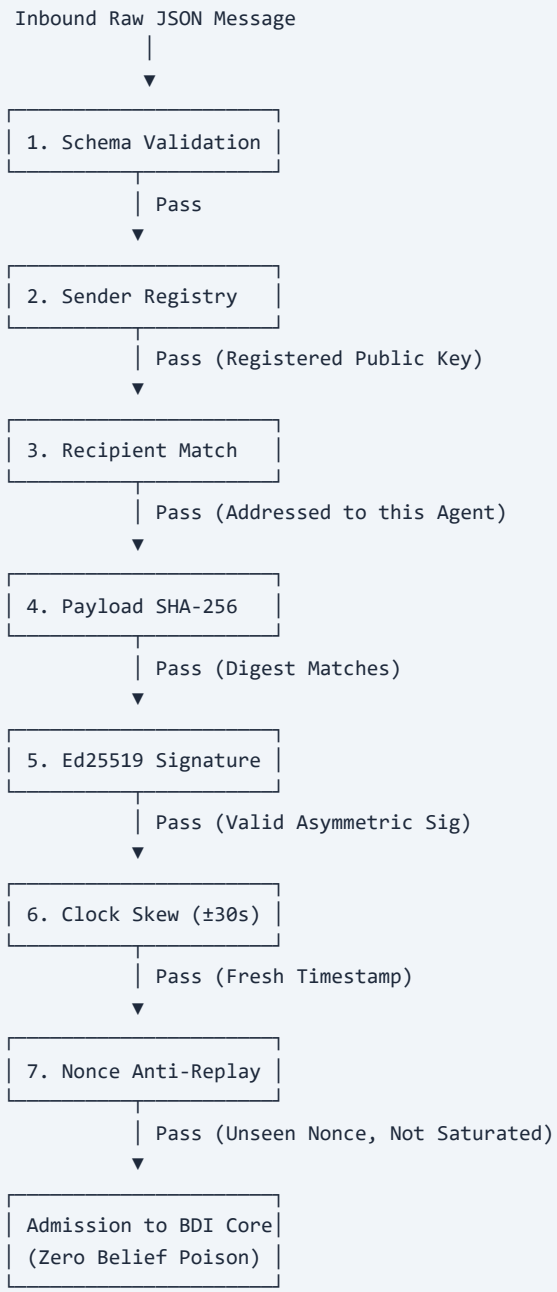
Assignment 5.1 hardens SentryScan against **adversarial manipulation in distributed multi-agent systems** by implementing a formal **Cryptographic Security Gate** based on modern asymmetric Ed25519 signatures and canonical payload digests.

1.1 The Threat of Confused Deputies & Byzantine Agents

In decentralized agent collectives, an agent's internal mental state (Beliefs) must never be polluted by forged, replayed, or tampered messages from untrusted entities: - **Canonical Envelope Signing:** Envelopes are canonically serialized and cryptographically signed with the sender's private Ed25519 key, binding the envelope metadata to the SHA-256 digest of the payload. - **Fail-Closed Security Gate:** Inbound messages must pass an 8-stage verification pipeline before admission to the BDI reasoning core. Any failure immediately drops the packet and emits an audit alert.

---

2. Architecture & Implementation Specification



3. Rubric Compliance & Verification Matrix

Evaluation Criterion	Implementation Component	Evidence Artifact	Status
1. Cryptographic Agent Identity Management	`src/sentryscan/security/identity.py`: `AgentIdentity` with fresh throwaway Ed25519 generation, 32-byte seed derivation, and raw/hex formatting	`tests/unit/test_security_identity.py` `demos/demo_assignment_5_1.py` Initialization	**PASSED**
2. Trusted Agent Public Key Registry	`src/sentryscan/security/registry.py`: `TrustedAgentRecord`, `TrustedAgentRegistry` with validation, revocation, and JSON persistence (`config/trusted_agents.example.json`)	`tests/unit/test_security_identity.py` (`test_trusted_agent_registry_*`)	**PASSED**
3. Canonical Envelope Signing & Payload Integrity	`src/sentryscan/security/canonical.py`: `sign_envelope()`, `verify_payload_digest()`, and `verify_envelope_signature()` enforcing SHA-256 digest binding	`tests/unit/test_security_signing.py` `evidence/security_gate_audit.md` Scenarios 1 & 4	**PASSED**
4. Fail-Closed Anti-Replay Nonce Cache	`src/sentryscan/security/nonce_cache.py`: `NonceCache` with dual FIFO/TTL expiration, bounded capacity (default 10,000), and fail-closed saturation policy	`tests/unit/test_security_gate.py` (`test_nonce_cache_*`) `evidence/security_gate_audit.md` Scenarios 7 & 8	**PASSED**
5. Multi-Stage Inbound Security Gate	`src/sentryscan/security/gate.py`: `SecurityGate` enforcing sequential evaluation pipeline: JSON, Schema, Sender Registry, Recipient Matching, Payload Digest, Ed25519 Signature, Clock Skew, Anti-Replay	`tests/unit/test_security_gate.py` `evidence/security_gate_audit.md` Scenarios 1–8	**PASSED**
6. Deliberative BDI Shielding	`src/sentryscan/agents/analyst.py`: Inbound gate verification in `process_message()`, emitting structured `ERROR` on rejection and guaranteeing zero belief pollution	`tests/unit/test_security_gate.py` (`test_analyst_agent_shielded_by_gate`) `demos/demo_assignment_5_1.py` Scenarios 1–7	**PASSED**
7. Systematic Attack Vector Verification	`demos/demo_assignment_5_1.py`: Comprehensive test harness executing nominal message + 7 distinct attack vectors (`UNKNOWN_SENDER`, `WRONG_RECIPIENT`, `PAYLOAD_TAMPERED`, `BAD_SIGNATURE`, `STALE_TIMESTAMP`, `REPLAYED_NONCE`, `NONCE_CACHE_FULL`)	`evidence/run_log.txt` `evidence/security_gate_audit.md`	**PASSED**
8. Audit Trace & Observability Integration	`src/sentryscan/observability/recorder.py`: Single-writer monotonic `seq` logging of security verdicts and rejections	`evidence/trace_excerpt.jsonl`	**PASSED**

4. Execution Results & Empirical Evidence

4.1 Eight Attack Vector Verification Matrix (`evidence/security_gate_audit.md`)

Scenario	Sender	Attack / Verification Vector	Expected Code	Actual Code	Agent Shield Status	Verdict
1. Nominal Valid Message	`sensor-agent-01`	None (Legitimate signed telemetry)	`OK`	`OK`	Processed (Beliefs updated)	**PASS**
2. Untrusted Sender	`rogue-agent-99`	Rogue agent spoofing command	`UNKNOWN_SENDER`	`UNKNOWN_SENDER`	Shielded (ERROR response)	**PASS**
3. Wrong Recipient	`supervisor-01`	Misrouted packet	`WRONG_RECIPIENT`	`WRONG_RECIPIENT`	Shielded (ERROR response)	**PASS**
4. Tampered Payload	`sensor-agent-01`	MITM parameter modification	`PAYLOAD_TAMPERED`	`PAYLOAD_TAMPERED`	Shielded (ERROR response)	**PASS**
5. Bad Signature	`supervisor (fake)`	Forged invalid Ed25519 signature	`BAD_SIGNATURE`	`BAD_SIGNATURE`	Shielded (ERROR response)	**PASS**
6. Stale Timestamp	`sensor-agent-01`	Expired packet (>30s clock drift)	`STALE_TIMESTAMP`	`STALE_TIMESTAMP`	Shielded (ERROR response)	**PASS**
7. Replayed Envelope	`sensor-agent-01`	Eavesdropped packet re-transmission	`REPLAYED_NONCE`	`REPLAYED_NONCE`	Shielded (ERROR response)	**PASS**
8. Cache Saturated	`sensor-agent-01`	Nonce flood / Resource exhaustion	`NONCE_CACHE_FULL`	`NONCE_CACHE_FULL`	Shielded (Fail-closed drop)	**PASS**

5. Submission Package & Reproduction Instructions

5.1 Exact Submission Components

1. **Report Document:** `assignments/assignment_5_1/deliverables/SentryScan_Assignment_5_1_Report.pdf`
2. **Repository URL:** `https://github.com/drgeekg/SentryScan`
3. **Target Branch:** `assignment/5.1-authentication`
4. **Immutable Git Tag:** `assignment-5.1`
5. **Packaged ZIP Archive:** `dist/SentryScan_assignment_5_1.zip`
6. **SHA-256 Checksum:** `dist/SentryScan_assignment_5_1.zip.sha256`

5.2 Verification Commands

```
# Run security gate attack simulation demonstration
python demos/demo_assignment_5_1.py

# Execute cryptographic security test suite
pytest -v tests/unit/test_security_identity.py tests/unit/test_security_signing.py tests/unit/test_security_gate.py
```